COL 7364/764 Assignment 3

Rank Fusion and Learning-to-Rank Arena

2026 – Version 1.5
(dated 15 October 2026)

Important Dates

This assignment has three key deadlines.

Phase	Description	Date
Competition Round	An open, continuously updated leaderboard. Public dev-set scores are visible in real time; private held-out scores are revealed at the deadline. You may submit your implementation as many times as you want during this phase. **Submissions will be made through a submission website accessible from the IITD network or VPN.**	**17 Oct 2026 11:59PM – 22 Oct 2026 11:59PM**
Conformance Freeze	The CI check must be green by this time. Interface failures reported for the first time after the freeze are graded as a submission error, not as a harness bug (Section 2.5).	**20 Oct 2026, 11:59 PM (48 hours before the competition deadline)**
Report Submission	Submit the report and statements described in Section 6. **Submit the report on Moodle.**	**24 Oct 2026 11:59PM**

The submission-server details will be posted on Piazza/Moodle when leaderboard submissions open.

Instructions

Follow all instructions. Submissions that do not follow these instructions will not be evaluated.

1. This programming assignment may be completed individually. Do not collaborate by sharing code, algorithms, or other pertinent implementation details. You may discuss the assignment and post questions seeking clarification. Please tag all Piazza discussions related to this assignment with `hw3`. Verify the exact tag on Piazza if a different tag has been announced.
2. All programs must be written in Python only. The evaluation machine runs Ubuntu 24.04.3 LTS, Python 3.12.3, 4 CPU cores, 8 GB RAM. We recommend using compatible versions and avoiding deprecated or unsupported functions.
3. Submit a single ZIP file containing the source code and your trained artifacts. When extracted, the ZIP file must reproduce the directory structure of the associated GitHub repository. Missing repository files or directories may cause the submission not to be evaluated. Do not remove or rename files already present in the repository; you may add files as needed.
4. Do not submit notebooks. All programs will be run from the command line only. Follow the file names, naming conventions, and function signatures specified below and in the associated repository.

5. The assignment will be built and run on Ubuntu Linux. Ensure that file names, paths, and argument handling are compatible with that environment.
6. Do not submit data files. If you plan to use any additional non-standard library, consult the instructor first or post a question on Piazza.
7. There will be no deadline extensions. This assignment requires substantial implementation effort, offline model training, and careful hyperparameter selection. Start early.

The directory structure in each submission must match the associated GitHub repository. Do not remove or rename files already present in the repository. You may add files as needed.

1 Assignment Description

The goal of this assignment is to **fuse** multiple base rankers, to implement **pointwise** and **pairwise** learning-to-rank models, and to train a **generalizing** LambdaMART model that does not overfit your training queries. In Assignments 1 and 2 you computed lexical signals yourself and reranked documents with them. Here you will instead combine signals that other systems have already computed, and learn a ranking function on top of them. These two skills are what take a search system from a single scoring function to the kind of ranker that production search engines actually run.

You will fuse several independently computed base rankers into a single ranking (no learning, just combination rules), and separately train a learning-to-rank (LTR) model—pointwise, then pairwise, then a gradient-boosted listwise model in the LambdaMART family—over a standard, published feature set. As in Assignment 2, the leaderboard does not just reward whichever model posts the best accuracy on the data you can see. A sufficiently powerful, sufficiently over-parameterized LTR model can fit a small training query set almost perfectly and still fail to generalize to a query it has never seen. This is a well-documented failure mode of learning to rank in practice, not a contrived one. Section 4 is designed so that a model which only memorizes cannot win.

1.1 Learning Objectives

- Implement unsupervised rank fusion—reciprocal rank fusion (RRF) and at least one score-based method (CombSUM or CombMNZ)—to combine multiple independently ranked lists into one, without any training data [CCB09, FS94].
- Train a pointwise learning-to-rank model (regression over individual query–document pairs) and a pairwise model (RankNet-style, trained on preference pairs), writing the loss functions and pair construction yourself on top of a standard autodiff library, to understand what each formulation does and does not capture about ranking [Liu09].
- Train a listwise, gradient-boosted ranking model in the LambdaMART family [Bur10, WBSG10] over a standard, published learning-to-rank feature set, and understand why it is the dominant practical approach in industrial ranking systems.
- Experience, hands-on, the gap between fitting a training query set and generalizing to queries never seen during training, and learn to recognize when a model's leaderboard-topping accuracy is a product of real ranking signal versus overfitting to a small, idiosyncratic training set.
- Understand a second concrete failure mode of single-condition evaluation, distinct from Assignment 2's query drift: a leaderboard that only ever reports accuracy on data a team could have tuned against rewards memorization exactly as readily as it rewards generalization, unless something explicitly measures the gap between the two.

1.2 Background

1.2.1 Rank Fusion

Rank fusion combines multiple ranked lists—produced by different retrieval or scoring systems, with no shared score scale and no training signal—into a single consensus ranking.

Reciprocal rank fusion (RRF) [CCB09] scores a document by summing $1/(c + \text{rank})$ across every list in which it appears, where c is a small constant (Cormack et al. use $c = 60$):

$$\text{RRF}(d) = \sum_{s \in S} \frac{1}{c + \text{rank}_s(d)}.$$

RRF needs only each system's rank positions, never their raw scores, which makes it robust to systems whose scores live on very different, incomparable scales.

CombSUM and *CombMNZ* [FS94] instead combine (typically min–max normalized) raw scores directly. CombSUM sums them; CombMNZ additionally multiplies the sum by the number of lists in which the document appears at all, rewarding documents that multiple systems agree on.

None of these methods uses relevance judgments. They are combination rules, not learned models, which is why they are this assignment's required baseline (Section 2.1). They play the same role that the unigram query-likelihood reranker played relative to RM3 in Assignment 2: a correct, well-understood floor that any learned model should be expected to clear.

1.2.2 Learning to Rank: Pointwise, Pairwise, and Listwise

Pointwise. Pointwise approaches treat ranking as ordinary regression or ordinal classification: predict a relevance score (or grade) for each query–document pair independently, then sort by the predicted score. This is the simplest formulation and the easiest to get right, but it optimizes the wrong objective. A model that predicts absolute relevance grades well can still produce a mediocre ranking, because a pointwise loss does not know that getting the top of the list right matters more than getting the bottom right [Liu09, Ch. 3].

Pairwise. Pairwise approaches train on preference pairs instead: for two documents D_1, D_2 under the same query with different grades, learn to predict which should rank higher. RankNet [BSR+05] is the canonical pairwise formulation, using a cross-entropy loss over predicted pairwise preference probabilities. Pairwise training directly optimizes relative order, which is closer to what ranking needs, but it still treats every pair as equally important. In reality, a swap near the top of the list (which affects nDCG@10 far more) should matter more than a swap near the bottom.

Listwise. Listwise approaches optimize a ranking metric directly over the whole list. LambdaMART [Bur10, WBSG10] is the dominant practical listwise method: it keeps RankNet's pairwise gradient formulation but reweights each pair's gradient (the "lambda") by how much swapping that pair would change nDCG, and then fits those lambdas using gradient-boosted regression trees (MART) instead of a neural network. This combination—pairwise-style gradients, listwise-aware reweighting, and tree-based learners—is why LambdaMART remains a standard baseline in industrial ranking systems, years after more exotic methods have come and gone.

1.2.3 Generalization, and Why a Dev-Only Leaderboard Would Reward Overfitting

LambdaMART-family models are powerful and easy to overfit. With enough trees, enough depth, and a training query set small enough to memorize, a gradient-boosted ranker can learn idiosyncrasies of the specific training queries—which document happened to rank first for one query's particular feature combination—rather than a ranking function that generalizes to new queries. This is an ordinary risk in applied learning to rank: industrial ranking teams routinely report strong training-set gains that evaporate, or reverse, on live traffic.

A leaderboard that only ever reports accuracy on data a team could have tuned hyperparameters, features, and early stopping against cannot distinguish a model that generalizes well from one that has simply memorized that data more thoroughly: both would post the same dev-set nDCG@10. Only a score computed on a query set

no team ever saw during model selection reveals the difference. This is why Track C (Section 4) exists, and it mirrors Assignment 2's point about clean-condition-only evaluation: a different mechanism (overfitting to training queries rather than drift from noisy feedback), but the same lesson about what a single-condition leaderboard can and cannot tell you.

2 Task Specification

2.1 What You Must Implement

You must build the following. No existing rank-fusion or learning-to-rank library may implement the core logic of the **required** components for you (for example, RankLib, PyTerrier's fusion transformers, or LightGBM's built-in ranker used as a black box for anything other than the explicitly permitted LambdaMART step below). You may use a standard numerical or machine-learning library (NumPy, pandas, scikit-learn's generic regressors) as a computational building block, the same way Assignment 1 allowed using your own data-structures library.

Rank Fusion (REQUIRED baseline). Exposed as `fuse_candidates()` (Section 2.5). Implement RRF and at least one score-based method (CombSUM or CombMNZ) over the small set of named base-ranker scores that the harness provides per candidate (Section 3). The fusion code must be hand-written; no fusion library may be used. You may weight channels, choose the normalization, and tune RRF's constant c, using TRAIN only. CombMNZ counts a channel toward a document's multiplier only when the document's min–max-normalized score in that channel is greater than 0.

Pointwise and Pairwise LTR (REQUIRED, hand-checkable progression). Train a pointwise model and a RankNet-style pairwise model on the released TRAIN split, using the same small neural scoring network for both, so that the only thing that differs between them is the loss function you write. You may use a standard deep-learning library (PyTorch, or equivalent) for tensors, automatic differentiation, and optimizers; you may not use any library's ready-made ranking loss. Section 2.3 spells out exactly what the starter repository provides, what you write, and how it is checked; no prior experience training neural networks is assumed. These two models are graded on correctness against small hand-checkable examples (Section 7), independently of leaderboard score, to demonstrate that you understand what the pointwise and pairwise formulations actually do. In addition, your trained pairwise model is scored on the leaderboard as Track P (Section 4, 10%), so the care you put into pair sampling and training pays off directly; the pointwise model is graded on correctness only.

LambdaMART (REQUIRED, competition entry). Used through a library, not implemented by you, and exposed as `rerank_with_ltr()`: a listwise, gradient-boosted ranking model in the LambdaMART family. You are not asked to write the NDCG-weighted gradient computation or the boosted-tree learner; you call a standard gradient-boosting library's ranking objective (LightGBM's `lambdarank`, XGBoost's `rank:ndcg`, or an equivalent) and are graded on how well you train and tune it: feature handling, number and depth of trees, learning rate, regularization, and how you pick them (the cross-validation procedure in Section 2.4). You are still expected to understand it conceptually—what a "lambda" is, why it is weighted by the change in nDCG from swapping a pair, and what the trees are fitting—and to explain this at the understanding check (Section 5). This is the only required component where an existing library's core ranking logic is permitted; the rank fusion and pointwise/pairwise components above are not covered by this exception.

Offline Training. Training happens offline, not during grading. You will submit a trained model artifact (Section 2.5) alongside your code; `prepare()` loads it. The harness never retrains your model; this keeps grading fast and deterministic, and matches how trained-model submissions work in practice. A `scripts/train_model.py` that you write and run yourself, against the released TRAIN split (Section 3), is how you produce that artifact; it is not run as part of grading Tracks A, B, P, and C2. The one exception is Track C1 (below and Section 4), where staff run your training recipe themselves.

Training Recipe for Undisclosed Benchmarks (required, Track C1). Staff hold two further learning-to-rank benchmarks whose identity, size, and feature meanings are not disclosed. They share MSLR's flat-feature

SVMLight format but have different, unnamed features, so a model trained on MSLR cannot be applied to them; what must transfer is your *method*. Submit `submission/recipe.py` exposing

```
train(train_path, vali_path, out_model_path, seed=0),
```

which reads a flat-feature file of unknown dimensionality and writes a LightGBM/XGBoost-family model that your own `prepare()` and `rerank_with_ltr()` can load and score. These functions must not hard-code 136 features or any feature index. Staff run your recipe once per benchmark, on that benchmark's train and vali files, in an isolated directory, and evaluate the model it writes on the benchmark's held-out split.

- **What you are told:** at most about 300 features; integer labels 0 to 4 (a benchmark may use fewer levels); at most about 500,000 training rows; files may carry trailing # comments.
- **Limits, per benchmark:** 20 minutes wall-clock on 4 CPU threads; no network access; no files other than the two inputs; deterministic given the seed. A recipe that fails, times out, or writes an unloadable model earns 0 on that benchmark.

Your recipe may adapt its hyperparameters to the data it sees (for example, tree count or leaf size as a function of data size, or cross-validation inside the time limit), which is exactly the skill being tested: choices made by cross-validation on MSLR TRAIN should transfer, not be memorized. `docs/TRANSFER_TRACK.md` and `scripts/run_recipe.py` (three synthetic transfer sets of different width and label range) let you check the mechanics locally.

2.2 What is a valid "Entry"

Your competition entry is `submission/ranking.py` — `prepare(model_path)`, `fuse_candidates(qid, candidate_doc_ids, base_scores, k)`, and `rerank_with_ltr(qid, candidate_doc_ids, features, k)` (exact signatures in the starter repo) — plus `submission/recipe.py` (the training recipe for the undisclosed benchmarks, Section 2.1), `submission/fusion.py` and `submission/pointwise_pairwise.py` (the hand-checkable components graded separately per Section 2.1) and three trained artifacts: `submission/model.txt` (the LambdaMART model, in LightGBM text format; this is the path the harness passes to `prepare()`), `submission/pointwise_model.pt` and `submission/pairwise_model.pt` (the two neural scorers, loadable with the starter's `load_scorer()`; staff use them for the check in Section 2.3, test (e)). As in Assignment 2, there is no build / load process-isolation step and no Docker image: the harness imports your module directly and calls these functions in-process, loading your pre-trained model rather than fitting anything at grading time.

2.3 Pointwise and Pairwise Models: Exactly What Is Expected

This section exists because not everyone in the class has trained a neural network before, and the pointwise/pairwise requirement should test whether you understand the two ranking formulations, not whether you already know deep-learning tooling. The starter repository includes `docs/RANKNET_PRIMER.md`: a short walk-through of one training step (forward pass, loss, backward pass, optimizer step) with a tiny worked numerical example, and a list of the failure modes beginners most often hit (unscaled features, learning rate too high, pairs formed across queries, forgetting to shuffle, forgetting to fix the random seed). Read it first.

	What it covers	Who writes it
Provided	Data loading and per-query grouping; feature standardization using TRAIN-set statistics (raw MSLR features sit on very different scales, and unscaled input is the most common reason training silently fails); a dev nDCG@10 evaluation helper; a small scoring network (a multilayer perceptron, 136 inputs $\to$ 64 hidden units with ReLU $\to$ 1 output score; you may change width and depth); and a training-loop skeleton with TODO markers.	Starter repository
You write	(1) The pointwise loss: squared error between the network's score and the graded relevance label; (2) `make_pairs()`: within-query preference pairs; (3) the RankNet loss; (4) the training-loop pieces marked TODO: minibatching, forward pass, loss, backward pass, optimizer step, per-epoch dev nDCG@10, early stopping on dev, fixed random seed.	You (roughly 100–150 lines in total)
Allowed building blocks	PyTorch (or TensorFlow/JAX): tensors, autograd, `nn.Linear`/`nn.Sequential`, activation functions, Adam/SGD optimizers, `DataLoader`, random-seed utilities.	—
Not allowed	Any library's ready-made ranking loss or LTR trainer (allRank, TF-Ranking, PT-Ranking, RankLib, and so on), and LightGBM/XGBoost for these two models. Training on anything other than relevance labels—in particular, distilling another model's scores (for example, LambdaMART's) into the network. Note that PyTorch's `MarginRankingLoss` is a hinge loss, not RankNet: you must write the logistic pairwise loss yourself.	—

Functions staff test by name. Keep these names and signatures in `submission/pointwise_pairwise.py` exactly as the starter declares them: `set_seed`, `pointwise_loss`, `make_pairs`, `ranknet_loss`, `pointwise_step`, `pairwise_step`, `train_model`, and the provided `load_scorer`. RRF, like CombSUM and CombMNZ, is yours to write; the starter leaves all three as TODOs, and its baseline `ranking.py` uses a stand-in (sort by the BM25 channel) until your `rrf_scores` works.

The RankNet loss. For a pair of documents under the same query where document i has a higher relevance grade than document j, the network produces scores s_i and s_j. RankNet models the probability that i should rank above j as

$$P(i \succ j) = \sigma(s_i - s_j),$$

and the loss for the pair is

$$-\log P(i \succ j) = \log\big(1 + \exp(-(s_i - s_j))\big).$$

Compute it in a numerically stable form (`softplus(-(s_i - s_j))` or `torch.logaddexp(0, -(s_i - s_j))`) rather than evaluating exp directly, and average over the pairs in the minibatch. Burges et al. [BSR+05] derive this from a probabilistic cost; the starter repository's primer walks through the same derivation in three lines.

Forming pairs. A query typically has on the order of 100 judged documents, which gives thousands of possible pairs per query—far too many to materialize across the whole TRAIN split. `make_pairs()` must (a) only ever pair two documents from the same query, (b) include only pairs whose relevance grades differ (ties carry no preference), and (c) orient each pair so the more relevant document comes first. Rather than building every pair up front, sample a fixed number of pairs per query at the start of each epoch (a few hundred per query is plenty; the starter skeleton suggests a default). Compute budget: everything runs on a laptop CPU, and a full training run should take minutes, not hours. If yours takes much longer, you are probably materializing too many pairs.

Minibatch training, step by step. An epoch is one pass over the pairs sampled for that epoch (not over every possible pair). A minibatch is a slice of those pairs; the starter default is 1,024. Training repeats the same five steps for each minibatch:

1. stack the feature rows of the batch's preferred documents into one tensor and those of its other documents into another, each of shape (batch size, 136);
2. run both through the same network (shared weights) to get score vectors s_i and s_j;
3. compute the mean RankNet loss over the batch;
4. call `loss.backward()` to compute gradients;
5. call `optimizer.step()` to update the weights, then `optimizer.zero_grad()`. Forgetting the last call is the classic beginner bug, because gradients silently accumulate across batches.

Reshuffle the pairs at the start of every epoch; if all of one query's pairs sit next to each other, each batch's gradient is biased toward that query. Why batches at all: one pair per step is too noisy, and all pairs per step is too slow and too large for memory; a few hundred to a few thousand is the usual middle. The pointwise model uses the identical loop with documents instead of pairs and squared error instead of the pairwise loss.

Suggested defaults, all adjustable: Adam optimizer, learning rate 10^{-3}, batch size 1,024, at most 20 epochs, and stop when dev nDCG@10 has not improved for 3 consecutive epochs. Early stopping on dev is allowed but counts as tuning against dev (Section 4): declare it, or carve a validation split out of TRAIN instead. Two cautions. First, a falling training loss does not mean the ranking is improving: the loss is a pairwise proxy, so track dev nDCG@10 after every epoch. Second, evaluation is not batched like training: to compute nDCG@10 you score all documents of a query (with gradients off and the network in evaluation mode), sort them, and then compute the metric. A pairwise batch has no notion of a ranked list.

Track P: the pairwise model on the leaderboard. Staff load your `submission/pairwise_model.pt` with their own copy of the starter's `harness/neural.py` and score it on the MSLR held-out split; its nDCG@10 is percentile-ranked (10% of the grade; dev scores appear on the live leaderboard as feedback only). To make it rank better, you may change, in your own additional functions or `train_model` options:

- how pairs are sampled (for example, favouring pairs with a large label gap or involving top-ranked documents);
- the number of pairs per query, epochs, learning rate, batch size, and feature preprocessing; and
- the network's width and depth: a plain MLP with at most 3 hidden layers of at most 256 units each.

You may also weight each pair, for example by the change in nDCG@10 from swapping the two documents, which turns RankNet into LambdaRank [Bur10]; write such weighting as a separate function, never inside the graded `ranknet_loss`. You may not train on anything other than relevance labels (no distillation), and you may not change the artifact format: an artifact that the staff loader rejects, or that breaks the size bound, scores 0 on Track P without affecting your other tracks. Declare, as for every model, which splits you trained and tuned on.

What is checked, and what is not. Staff run hand-checkable tests (Section 7, "Correctness of required components"):

(a) your RankNet loss, evaluated on tiny score/label examples with known answers;

(b) `make_pairs()` returns only same-query, strictly ordered pairs;

(c) one gradient step on a single pair increases $s_i - s_j$, i.e. the preferred document's score moves up relative to the other;

(d) the pointwise loss matches squared error on a toy example;

(e) a basic "learns something" floor: your pairwise model's dev nDCG@10 beats simply sorting by a single BM25 feature, a threshold set low enough that a correct implementation clears it comfortably; and

(f) training is reproducible given the seed.

These checks test the textbook functions, so keep them textbook: `ranknet_loss` stays the plain, unweighted RankNet loss even if you train your leaderboard model differently. You are not expected to tune the pointwise model at all; the pairwise model is also scored as Track P (next paragraph). In your report (Section 6), compare pointwise, pairwise, and LambdaMART dev-set nDCG@10 honestly, whichever way the numbers come out. LambdaMART will usually win, but what you observe is what you report.

2.4 Cross-Validation and Hyperparameter Selection

You do not have the held-out set, so cross-validation within the released TRAIN split is how you estimate, locally, how much your model loses on queries it was not trained on (Section 6, deliverable 2), and how you choose LambdaMART hyperparameters without repeatedly tuning against dev. The procedure:

1. Split by query, never by row: all documents of a query go to the same fold. Splitting by row puts near-duplicate documents of the same query in both training and validation and inflates the validation score, which is exactly the overfitting you are trying to measure. The starter repository provides `grouped_kfold(qids, k=5, seed=0)` so that you do not have to get this subtle part right yourself.
2. For each fold in turn, treat it as validation and the other four as training.
3. Fit the feature standardizer on the four training folds only; computing means and standard deviations over all the data first leaks information from the validation fold.
4. Train on the four folds, then compute mean nDCG@10 over the queries of the validation fold, and also over the training folds the model just saw.
5. Report the mean and standard deviation of the validation score across folds, and the ratio of mean validation nDCG@10 to mean training-fold nDCG@10: your local estimate of how much a model loses on queries it was not trained on.
6. Do not early-stop on the validation fold you are about to report: that quietly turns it into training data for the stopping decision and biases the estimate upward. Treat the number of trees (LambdaMART) or epochs (neural models) as a hyperparameter that you sweep over fixed values, and report the validation score at each.

Cross-validation is required for your LambdaMART sweep (it trains in seconds to minutes). For the pointwise and pairwise networks a single train/dev run is sufficient, so you are not asked to train each neural model five times. Once you have chosen hyperparameters, retrain once on the full TRAIN split and ship that model.

2.5 Submission Interface and Conformance

Start from the starter repository. The starter repository ships a trivial, fully working baseline (RRF fusion and a tiny pre-trained LambdaMART model) that runs end-to-end through the harness. Fork it, keep the same shape, and replace the internals.

Implement the three entry points. Your competition entry is `submission/ranking.py`, which must expose:

```
prepare(model_path: str) -> None

fuse_candidates(qid: str, candidate_doc_ids: List[str],
                base_scores: Dict[str, Dict[str, float]],
                k: int) -> List[Tuple[str, float]]
```

```
rerank_with_ltr(qid: str, candidate_doc_ids: List[str],
                features: Dict[str, List[float]],
                k: int) -> List[Tuple[str, float]]
```

- `prepare()` loads your trained model artifact. It must load without retraining: a `prepare()` that silently falls back to training from scratch when the artifact is missing or corrupt is treated as a conformance failure, not as an acceptable fallback.
- `fuse_candidates()` receives `base_scores`, which maps each candidate document ID to a small dictionary of named base-ranker scores (Section 3).
- `rerank_with_ltr()` receives `features`, which maps each candidate document ID to its full 136-dimensional feature vector.
- Both ranking functions may return only document IDs from the candidate pool they were given, exactly as in Assignment 2. The harness checks this directly, using the same anti-gaming mechanism.

Include the other required files. In addition to `submission/ranking.py`, your submission must contain:

- `submission/fusion.py` and `submission/pointwise_pairwise.py`, the hand-checkable components graded separately (Section 2.1);
- `submission/recipe.py`, the training recipe for the undisclosed benchmarks (Section 2.1); and
- three trained artifacts: `submission/model.txt` (the LambdaMART model, in LightGBM text format; this is the path the harness passes to `prepare()`), and `submission/pointwise_model.pt` and `submission/pairwise_model.pt` (the two neural scorers, loadable with the starter's `load_scorer()`; staff use them for check (e) in Section 2.3, and the pairwise model for Track P).

How the harness runs your submission. As in Assignment 2, there is no build/load process-isolation step and no Docker image: the harness imports your module directly and calls these functions in-process, loading your pretrained model rather than fitting anything at grading time. This keeps grading time bounded and deterministic, which is the whole point of the offline-training split (Section 2.1). The following wall-clock caps are enforced:

- `prepare()` (model loading only): at most 60 seconds;
- each `fuse_candidates()` or `rerank_with_ltr()` call: at most 2 seconds;
- the whole grading run: at most 30 minutes.

The caps are bounded by candidate pool size, not dataset size, and are restated in `docs/GRADING.md`.

The one exception: your training recipe. For Track C1 only, staff do run training: they run your `submission/recipe.py` on each undisclosed benchmark and score the model it writes with your own `prepare()` and `rerank_with_ltr()`. The limits and rules for this run are given in Section 2.1.

Check conformance before the freeze. Every push runs the public harness on the toy set and reports pass/fail within minutes. 48 hours before the deadline, this CI check must be green. Interface failures reported for the first time after the freeze are graded as a submission error (Section 7), not as a harness bug.

3 Dataset and Tools

Dataset (MSLR-WEB10K). The assignment uses MSLR-WEB10K [QL13], Microsoft's standard public learning-to-rank benchmark: 10,000 queries, each with a candidate pool of judged query–document pairs. Each pair is represented by a 136-dimensional named feature vector (query-term coverage, stream length, IDF, term frequency, BM25 and LMIR.JM/LMIR.DIR language-model scores—each computed per document field—plus link-graph features such as PageRank) and a 5-grade relevance label (0 = irrelevant to 4 = perfectly relevant).

Splits. We use the dataset's official Fold 1 split. Its *train* partition is released to you as `data/train/` (with labels, for fitting your models). Its *vali* partition is released as `data/dev/` (with labels, for local tuning and the public leaderboard; you may train on it, but read the note in Section 4 first). Its *test* partition is withheld as the MSLR held-out set, used for Track P and as the source of Track C2's hidden versions (Section 4); course staff evaluate your submission against it directly, and you never see it.

Base-ranker scores. The `base_scores` passed to `fuse_candidates()` are a small named subset of the full feature vector: several independently well-understood scoring functions (for example, BM25 on the whole-document field, BM25 on the title field, and LMIR.JM and LMIR.DIR on the whole-document field) that disagree with each other often enough that fusing them is a real exercise, not a formality. The exact feature indices are documented in `harness/feature_names.py`, transcribed from MSLR-WEB10K's official feature-definition table and verified by staff against the real data before release.

Document IDs. MSLR files carry no document identifiers, so the harness assigns each candidate the opaque ID `<qid>:<row within the query>`. Treat IDs as opaque strings.

Toy subset. A small subset (`data/toy/`, a few queries and candidates in the same file format) is provided for fast local iteration and is what the conformance CI runs against, as with the toy corpora in Assignments 1 and 2.

Two undisclosed benchmarks (Track C1 only). These are staff-held: you never receive them, and you are not told what they are. Only the LTR path is scored on them (no base-ranker channels are defined for them). See Section 2.1 for what you are told, and `docs/TRANSFER_TRACK.md` for the local dry run.

On Held-Out Secrecy — Read This

MSLR-WEB10K is a complete, publicly downloadable, fully labeled dataset. Unlike Assignment 2's genuinely undisclosed held-out topics, nothing prevents you from downloading the official archive yourself and looking up the true relevance labels for every row, including whichever rows make up the held-out set used for Track C2. The same holds for the held-out splits of the undisclosed benchmarks used for Track C1, and you must not try to identify those benchmarks. **Consulting official MSLR-WEB10K labels, or the labels of any public learning-to-rank dataset you suspect to be an undisclosed benchmark, for any row, by any means, for the purpose of this assignment, is an academic-integrity violation**, identical in kind to consulting a solution key. The held-out set's secrecy here is a matter of course policy, not technical enforcement; treat it accordingly (Section 8).

4 Competition Mechanics and Scoring

Format. The competition uses an open leaderboard with unlimited submissions before the deadline. Public dev-set scores are visible in real time, and private held-out scores are revealed at the deadline. As in Assignments 1 and 2, every accuracy-bearing component below is percentile-ranked against the whole class before weighting, not scored on raw metric values. For Tracks A and B, where teams cluster, metrics are first rounded to the nearest 0.005, so teams whose scores fall in the same band share a rank, and differences below the evaluation noise earn nothing.

Track	What is measured	Weight
A — Fusion baseline	nDCG@10 (80% of the track) and MAP@10 (20%) of `fuse_candidates()` on the dev set, each percentile-ranked against the class. MAP@10 treats label > 0 as relevant. Tied scores share the average percentile. Metrics are first rounded to the nearest 0.005, so near-ties share a rank.	10%
B — LTR dev accuracy	nDCG@10 of `rerank_with_ltr()` on the dev set, rounded to the nearest 0.005 and then percentile-ranked.	10%
P — Pairwise model	nDCG@10 of your `submission/pairwise_model.pt` on the MSLR held-out split, loaded with the staff copy of `harness/neural.py` (plain MLP, at most 3 hidden layers of at most 256 units), percentile-ranked. An artifact that does not load scores 0. Dev scores are shown live as feedback only.	10%
C1 — Transfer to undisclosed benchmarks	Staff run your `submission/recipe.py` on each of two undisclosed benchmarks and evaluate `rerank_with_ltr()` on each one's held-out split. Each benchmark's nDCG@10 is percentile-ranked separately, and the two percentiles are averaged. Failure or timeout on a benchmark scores 0 there.	35%
C2 — Hidden robustness test	Staff run your submitted model—the same `rerank_with_ltr()` as Track B, with no retraining—on five hidden versions of the MSLR held-out split. The mean nDCG@10 over the versions is percentile-ranked. How the versions are constructed is not disclosed.	35%

How the tracks are built. There is no bonus track: the leaderboard weights sum to 100%, with Tracks A, B, P, C1, and C2 at 10%, 10%, 10%, 35%, and 35%. Tracks A and B are weighted lightly on purpose: they are measured on dev data that every team can see and tune against, so strong teams cluster there, and they carry less of the grade than the three tracks computed on data you never see. Track P rewards the part of the LTR work that is your own code—pair sampling and training of the pairwise model—rather than library tuning. Track C1 asks a question a single-dataset leaderboard cannot: does your method transfer? Because the benchmarks have different, unnamed features, a model trained on MSLR is useless there and you cannot tune to them, so you cannot memorize your way to a good score; the only thing that carries over is how you set up and regularize training. Averaging two benchmarks reduces the luck of a single draw.

Track C2 asks a different question of the model you have already submitted: how well does it hold up on a hidden test it was never tuned for? Tracks C1 and C2 are both raw accuracy percentiles on data you never see, not ratios, so a trivial or badly under-trained ranker scores low rather than high, and neither can be raised by tuning against dev. Because the C2 test is hidden, there is nothing specific to prepare for: the best preparation is a well-regularized model whose accuracy does not depend on idiosyncrasies of the data you trained and tuned on.

Training on the Dev Set Is Allowed — Read What That Does and Does Not Cost You

The dev labels are released, so nothing technically stops you from adding the dev queries to your training data, or from tuning many hyperparameters, features, or stopping points against them. If you do, your dev-set nDCG@10 rises because the model has now seen those exact queries, and so do your Track A and B numbers on the dev leaderboard (20% of the grade). Your accuracy on queries you have not seen does not rise by anything like the same amount, so your public leaderboard position will overstate where you stand; Tracks P, C1, and C2 are computed on data you never see and gain nothing from it.

No scored track penalizes this directly, which is a limit of the scoring, not a licence: staff compare each team's dev score with its held-out score as a diagnostic, and the rules below are enforced. State in your README and report exactly which splits each submitted model was trained on, and which you used for tuning and early stopping; an undisclosed use of dev for training is a violation of the declaration rule in Section 8, not a modelling choice, and the understanding check (Section 5) will ask. If you do train on dev, you cannot use dev for cross-validation estimates, since it is no longer unseen; use grouped k-fold within the data you trained on (Section 2.4).

Baseline you must beat. A reference RRF-fusion-plus-LambdaMART implementation, tuned by course staff with undisclosed hyperparameters, is evaluated on the same tracks. Beating it is the pass/fail floor; your percentile rank across the tracks is what actually separates grades above that floor.

5 Understanding Check

A 10-minute walkthrough: explain your fusion method choice, your LambdaMART hyperparameters (tree count, depth, learning rate, early-stopping criterion) and why you chose them, and a live question in which course staff show you a dev-set query where your model's prediction and the true relevance order disagree, and ask you to reason about why, using your pointwise and pairwise models (Section 2.1) to help explain the disagreement. As in prior assignments, failing to explain or meaningfully reason about your own submission caps the leaderboard-performance component at 50% of its earned value.

6 What Must Be Submitted After the Competition?

1. Source code implementing the fusion, pointwise, pairwise, and LambdaMART models, plus your training recipe (`submission/recipe.py`), with a README describing your fusion method, your feature engineering (if any, beyond the 136 provided features), and your LambdaMART hyperparameters.
2. A short report (2–3 pages) containing:
   - a table or plot comparing pointwise, pairwise, and LambdaMART dev-set nDCG@10;
   - a plot of dev-set nDCG@10 versus at least one LambdaMART hyperparameter you swept (tree count or depth); and
   - a discussion of the dev-versus-held-out generalization gap you observed locally (using the grouped k-fold cross-validation procedure in Section 2.4 within the released TRAIN split, since you do not have access to the real held-out set) and what regularization choices you made in response.
3. A one-paragraph description of your final model configuration and why you settled there—specifically, what you traded off between dev-set accuracy and expected generalization.
4. An AI-use disclosure statement and a code-provenance statement (templates in the starter repository; see the course overview document, Section 4).

7 Grading Policy

Component	Weight	Criteria
Leaderboard performance	40%	The Section 4 leaderboard score (Tracks A + B + P + C1 + C2), scaled by percentile rank among the class on the private held-out set; clearing the undisclosed baseline is a pass/fail floor. Capped at 50% of earned value if the understanding check (Section 5) indicates the team cannot explain its own submission.
Understanding check	15%	The team correctly explains fusion and LambdaMART hyperparameter choices and can reason about a live disagreement between its model's prediction and the true relevance order.
Correctness of required components	20%	Your required fusion methods, pointwise loss, `make_pairs()`, and RankNet loss are each correctly implemented and independently verifiable against the small hand-checkable examples described in Sections 2.1 and 2.3, not leaderboard score alone. LambdaMART's use of a permitted gradient-boosting library does not substitute for correct pointwise/pairwise implementations.
Report quality	15%	The generalization-gap analysis is specific and evidenced (real plots from your own cross-validation runs), not generic or asserted without data.
Interface conformance	5%	CI smoke test passing at the conformance freeze; the submission runs unmodified through the harness, including the pretrained-artifact-only check (Section 2.5).
Code quality and reproducibility	5%	`train_model.py` reproduces the submitted model artifact from the released TRAIN split; `rerank_with_ltr()` is deterministic given the same inputs, and `recipe.train()` is deterministic given its seed.

8 Academic Integrity

No existing rank-fusion library may be used for `fuse_candidates()`, and no existing learning-to-rank library may implement your pointwise or pairwise models' core training logic (Section 2.1). The one named exception is LambdaMART itself: a standard gradient-boosting library's ranking objective (LightGBM `lambdarank`, XGBoost `rank:ndcg`, or an equivalent) is used for `rerank_with_ltr()`. You are not asked to implement LambdaMART, only to train, tune, and understand it, and this exception does not extend to the other required components.

Held-out labels. On the held-out set specifically (Section 3): consulting official MSLR-WEB10K relevance labels for any row—through the official download, a mirror, a cached copy from a prior course offering, or any other means—for the purpose of this assignment is an integrity violation, treated identically to consulting a solution key. This applies even though the data is technically public: the violation is in the use, not the access. Course staff may rotate which fold and which split serve as the held-out set between offerings for exactly this reason.

Declaring your training data. Your README and report must state which splits (TRAIN, DEV, or both) each submitted model was trained on, and which split you used for hyperparameter tuning and early stopping. Training on dev is permitted (Section 4 explains what it does and does not cost you); concealing it is not.

LLM use and provenance. As in prior assignments, LLM assistance is permitted but must be declared in an AI-use disclosure statement, and any external code or formula consulted beyond lecture material must be listed in a code-provenance statement. Course staff run similarity detection across current- and prior-term submissions;

a team that cannot explain flagged code at the understanding check is treated as an integrity case, not merely graded down.

References

[BSR+05] Christopher J. C. Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to Rank Using Gradient Descent. In *ICML*, pages 89–96, 2005.

[Bur10] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An Overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

[CCB09] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *SIGIR*, pages 758–759, 2009.

[FS94] Edward A. Fox and Joseph A. Shaw. Combination of Multiple Searches. In *TREC-2*, pages 243–252, 1994.

[Liu09] Tie-Yan Liu. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval*, 3(3):225–331, 2009.

[QL13] Tao Qin and Tie-Yan Liu. Introducing LETOR 4.0 Datasets. arXiv:1306.2597, 2013.

[WBSG10] Qiang Wu, Christopher J. C. Burges, Krysta M. Svore, and Jianfeng Gao. Adapting Boosting for Information Retrieval Measures. *Information Retrieval*, 13(3):254–270, 2010.